The Nature of CS

Subhashis Banerjee

Department of Computer Science
Ashoka University

TOC Class
March 18, 2026

Early algorithms

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times \underline{(n-1)!} & \text{otherwise} \end{cases}$$

$O(n)$ multiplications. Euclid's *Elements*. 300 BC.

$$gcd(m, n) = \begin{cases} m & \text{if } n = 0 \\ \underline{gcd(n, m \ mod \ n)} & \text{otherwise} \end{cases}$$

$O(\log n)$ steps. Euclid's *Elements*. 300 BC.

Early algorithms

$$x^n = \begin{cases} 1 & \text{if } n = 0 \\ x \times \underline{x^{n-1}} & \text{otherwise} \end{cases}$$

$O(n)$ multiplications. Dates back to the Egyptians. 2000 BC.

$$x^n = \begin{cases} 1 & \text{if } n = 0 \\ x \times sqr(\underline{x^{n/2}}) & \text{if } odd(x) \\ sqr(\underline{x^{n/2}}) & \text{if } even(x) \end{cases}$$

$O(\log n)$ multiplications. Acharya Pingala in *Chandah Sutra*. 200 BC.

Computation through the ages

Euclid, Archimedes, Eratosthenes, Pingala:

Aryabhatta, Bhaskara: integer solutions for linear Diaphontine equations. Led to the extended-Euclid for *gcd*.

Bhaskara/Jayadeva: The *chakravala* method (cyclic algorithm) for indeterminate quadratic equations

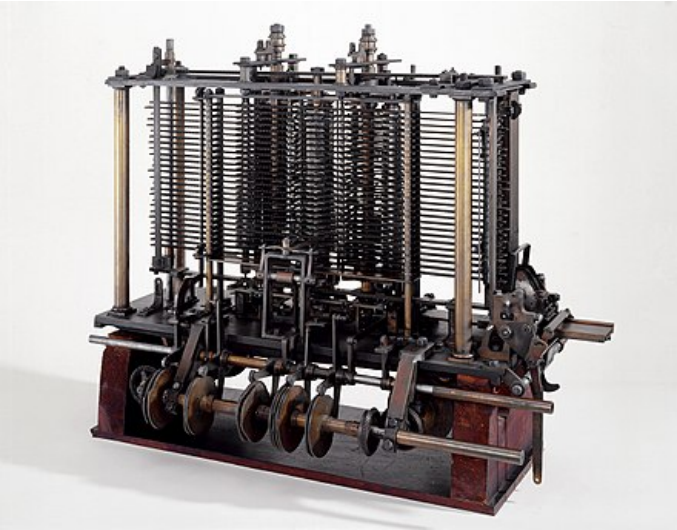
Newton: root finding, steepest descent, Newton's optimization

Gauss: multiplication in $O(n^{\log 3})$ and $O(n \log n)$. Integration (Gaussian quadrature), LU decomposition, least squares, regression, improved sieve, Gauss-Newton.

Many many others:

Towards universality

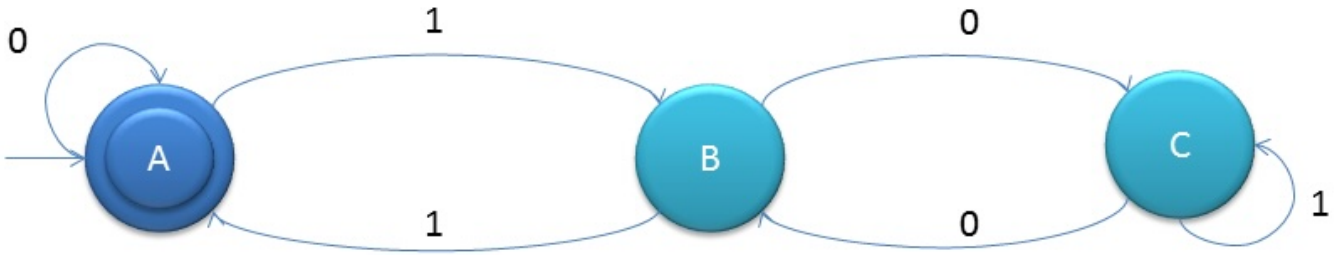
the Analytic Engine: Charles Babbage and Lady Ada Augusta (1837)



Z2,Z3: Konrad Zuse (1941)

Various mechanical computers: Leibnitz, Pascal,...

A finite state machine: the simplest computational model



0	1	10
11	100	101
110	111	1000

Context-Free Grammar (CFG)

Definition:

A *context-free grammar (CFG)* is a 4-tuple

$$G = (V, \Sigma, R, S)$$

where:

- V: finite set of **nonterminals**
- Σ: finite set of **terminals**, with $V \cap \Sigma = \emptyset$
- R: finite set of **production rules** of the form: $A \to \alpha$ where $A \in V$, $\alpha \in (V \cup \Sigma)^*$
- $S \in V$: **start symbol**

Language generated: $L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$

Key property:

- Each rule rewrites a *single nonterminal*, independent of context.

Example:

$$S \to 0S1 \mid \epsilon \quad \Rightarrow \quad L(G) = \{ 0^n 1^n \mid n \geq 0 \}$$

General models of computation

Recursive functions: Inductively defined functions $f : \mathbb{N}^n \to \mathbb{N}$

RAM model: Any programming language that supports **assignment**, **if-then-else**, **while-do**, an infinite array, 0 and $s \leftarrow s + 1$.

Turing machine: A mathematical model due to Alan Turing (1936). Consists of an **infinite tape**, a **finite state control**, a **read-write head** and **a program**.

Circuit model: *Acyclic* logic circuits of n input bits consisting of *NAND*, *FANOUT* and *CROSSOVER*; whose description can be generated by a *Turing machine*.

Computability and decidability

Hilbert's program: 30 problems (1905)

Gödel's incompleteness: In a logical system as powerful as *ordinary arithmetic* there must exist well-formed statement P such that neither P is a theorem nor $\neg P$ is a theorem. (1933)

Entscheidungsproblem: Logical version of Hilbert's 10th. Turing showed *undecidable* (1936).

The *Halting problem*

- **Input**: A string P and a string I. Think of P as a program.
- **Output**: 1, if P halts on I, and 0, if P loops forever on I.

Suppose $Halt(P, I)$ exists. Consider,

```
Program Z (String x) {
If Halt(x, x) then
Loop Forever
Else Halt
}
```

Consider program Z on input Z!

The halting problem is undecidable

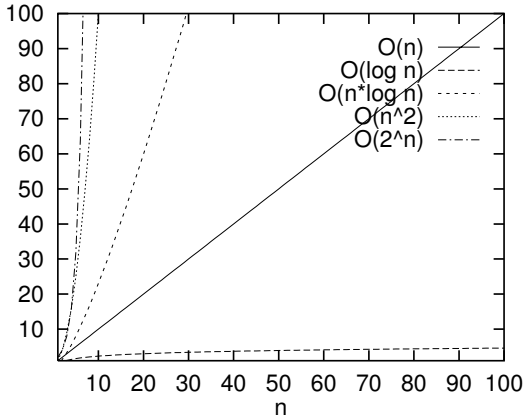
Church-Turing thesis

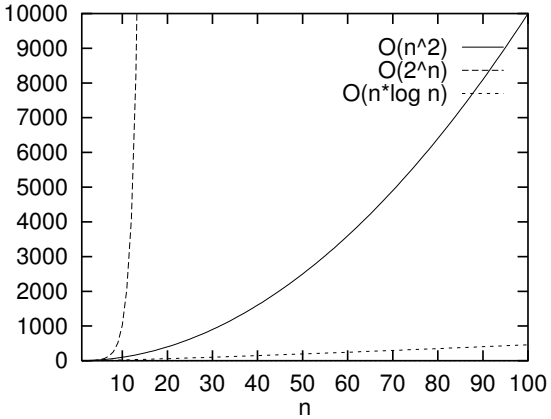
- All reasonable models of computations have turned out to be equivalent in terms of what they can compute.
- There can be a **Universal Turing machine** which can be used to simulate any Turing machine.
- The Universal Turing machine completely captures what it means to perform a computational task by algorithmic means.

The above has led to the assertion called the **Church-Turing thesis:**
If an algorithm can be performed on any piece of hardware (including a modern computer) then there is an equivalent algorithm for a Universal Turing machine which performs the same task.

What about efficiency?

- Roughly speaking, an efficient algorithm is one which runs in time **polynomial** in the size of the input.
- In contrast, an inefficient algorithm takes super-polynomial (typically exponential) time.



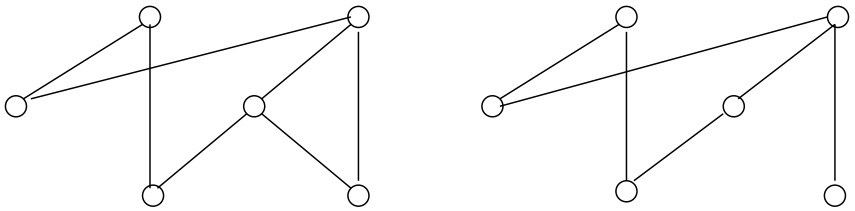


- Strengthened version of **Church-Turing thesis:** Any algorithmic process can be simulated **efficiently** using a Turing machine.

Decision problems and complexity classes

Decision problems:

- Given a composite integer m and $l < m$, does m have a non-trivial factor less than l?
- Does a given graph have a Hamiltonian cycle?



Complexity classes:

- **P** is the class of decision problems that a UTM can solve in polynomial time.
- **NP** is the class of decision problems whose solutions a UTM can verify in polynomial time.

NP-Hardness

Polynomial-time (Karp) reduction:

$$A \leq_p B \iff \exists f \in \mathrm{FP} \text{ such that } x \in A \iff f(x) \in B.$$

Definition (NP-hard):
A language (decision problem) H is **NP-hard** f every problem $L \in \mathrm{NP}$ can be polynomial-time reduced to H :

$$\forall L \in \mathrm{NP}, \quad L \leq_p H.$$

Interpretation:

- H is at least as hard as every problem in NP.
- If $H \in \mathrm{P}$, then $\mathrm{P} = \mathrm{NP}$.

Remarks:

- H need not be in NP.
- H may even be undecidable.

NP-Completeness

Definition (NP-complete):
A language C is **NP-complete** if:

$$\text{(i) } C \in \mathrm{NP},$$
$$\text{(ii) } \forall L \in \mathrm{NP},\ L \leq_p C.$$

Equivalently:

$$C \text{ is NP-complete} \iff C \in \mathrm{NP} \cap \text{NP-hard}.$$

Key property:

- If any NP-complete problem is in P, then:

$$\mathrm{P} = \mathrm{NP}.$$

Proof strategy (Cook–Levin paradigm):

- Show $C \in \mathrm{NP}$
- Show a known NP-complete problem (e.g., SAT) reduces to C

Does "coin toss" help?

- Consider a function $f : \{0, \ldots, 2^n - 1\} \rightarrow \{0, 1\}$.
- Suppose we are given that $f(x)$ is either *constant* (0 or 1 for all values of x) or *balanced* (0 for exactly half for all possible x and 1 for the other half).
- Our problem is to decide what type f is?
- Clearly, any deterministic algorithm will take at least $2^{n-1} + 1$ queries in the worst case.
- Alternatively, we can choose k (fixed) values of x *uniformly at random*. If $f(x)$ is different for any two conclude *balanced*, else conclude *constant*. In the later case there is a non-zero probability of error, equal to 2^{-k}.
- The probability bound is arbitrary. *Chernoff bound* can be used to amplify the probability to near 0 with only a few (logarithmic) repetitions.

Ranomized algorithms

Solovay and Strassen (1977)

$$prime(n,k) = \begin{cases} true & \text{if } n = 2 \\ prime_test(n,k,false) & \text{otherwise} \end{cases}$$

$$prime_test(n,k,failed) = \begin{cases} true & \text{if } k = 0 \\ false & \text{if } failed \\ prime_test(n,k-1,\neg Fermat_test(n)) & \text{otherwise} \end{cases}$$

$$Fermat_test(n) = (expmod(a,n,n) = a)$$

Randomized algorithms

- Solovay and Strassen showed, in mid 1970's, that a *randomized algorithm* could determine whether a number n is a prime (with an arbitrarily low probability 2^{-k}) or a composite (with certainty) in $O(k \log^3 n)$ time.
- *No efficient deterministic algorithm was known for the problem till Agarwal-Kayal-Saxena (AKS) in 2003.*
- Strengthened version of **Church-Turing thesis:** Any algorithmic process can be simulated **efficiently** using a *probabilistic* Turing machine.
- **BPP** is the class of problems that can solved *efficiently* using a probabilistic TM.

What is (not) known about complexity?

- Some other complexity classes: **L**, **PSPACE**, **EXP**.
- It is known that $\mathbf{L} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXP}$.

  $\boxed{\text{Is } \mathbf{P} = \mathbf{NP}?}$
- It is also known that $\mathbf{P} \subset \mathbf{EXP}$ and $\mathbf{L} \subset \mathbf{PSPACE}$. Hence at least one of the inclusions above must be strict. Which one?
- Also, clearly, $\mathbf{P} \subseteq \mathbf{BPP}$
- If an **NP**-Complete problem can be solved in time t, then all problems in **NP** can be solved in time $poly(t)$.

Where does quantum computing fit?

- Peter Shor (1994) gave an $O(n^3)$ quantum algorithm for *factoring* an n bit number. The best known classical algorithm for the problem is *number field sieve* which works in $exp(O(n^{1/3} \log^{2/3} n))$.
- Lov Grover (1995) gave an $O(\sqrt{n})$ quantum algorithm for *search* in an *unstructured search space* of size n.
- If an $O(\log n)$ algorithm could be found for search it would have established that **NPC** problems can be solved efficiently on quantum computers.
  Not to be - Grover's algorithm has been proved to be optimal.
- It is known that $\mathbf{P} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE}$.
- $\boxed{\text{Is } \mathbf{P} \subset \mathbf{BQP}?}$.

What about data?

- Regression
- Classification
- Clustering
- Topic discovery
- Generative models
- Causality

Can ML and data analytics lead to a post theory world?

- Can inductive inference lead to complete understanding?
- Soundness and completeness?
- Galileo Galeilei, regression and the $ut + \frac{1}{2}gt^2$ law?
- Classification: how many training examples do we need to produce H using a poly-time algorithm such that $\text{Prob}[C \triangle H < \epsilon] \geq 1 - \delta$?
- Distribution shift in the wild?

Two theories to be mindful of

Limitations of observation

In general there can be multiple joint probability distributions (over latent and observable variables) that can produce the same marginal distributions over the observables. Thus it is impossible to make causal inference from purely observational data.

Bias-variance tradeoff

- Suppose $y = f(x) + \epsilon$
- Estimate $\hat{f}(x; D)$ given $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$.
- For any $\hat{f}$,
  $$E_D\left[\left(y - \hat{f}(x, D)\right)^2\right] = \left(\text{Bias}_D\left[\hat{f}(x; D)\right]\right)^2 + \text{Var}_D\left[\hat{f}(x; D)\right] + \sigma^2$$

where,

- $\text{Bias}_D\left[\hat{f}(x; D)\right] = E_D\left[\hat{f}(x; D)\right] - f(x)$, and
- $\text{Var}_D\left[\hat{f}(x; D)\right] = E_D\left[\left(E_D\left[\hat{f}(x; D)\right] - \hat{f}(x; D)\right)^2\right]$.

Generative models

GPT-3 and ChatGPT

- Semi-supervised learning using 45 Terabytes of text data
- 175 billion internal parameters, but doesn't overfit.
- Prompt engineering
- Given a new prompt computes the probability of next word to output
- Context flows in from 'attention mechanism' in the transformer.

Deductive reasoning?

Definition

Given premises $P_1, P_2, \ldots, P_n$. Can a ChatGPT like system conclude C?

- Can it learn, purely inductively, the rules of an axiomatic system like ordinary arithmetic?
- Can it prove theorems in modern algebra?
- Can it write computer programs effectively?

Deductive reasoning?

- Any problem of propositional logic can be solved in exponential time (by truth tables). Evaluation in linear time.
- It is not known whether problems of propositional logic can be solved in polynomial time.

Time hierarchy theorem

There are an infinite hierarchy of problems that a time-bounded computation on Turing machines cannot solve. For example, there are problems that can be solved in $O(n^2)$ but not in $O(n)$.

$$DTIME\left(o\left(\frac{f(n)}{\log f(n)}\right)\right) \subsetneq DTIME(f(n))$$

- *Deduction problems have graded complexity. ChatGPT is a fixed time Turing machine (FSM?).*
- However, classical *resolution* and *search* can possibly be integrated.

Abductive reasoning

- Given an observation q, hypothesize p such that $p \implies q$.
- Selective abduction (selecting from a set of known rules) follows from inductive learning. Indeed, ChatGPT gives very reasonable alternatives for

Example

Lipika is usually at home on Saturdays but today she is in the CS department. What may have happened?

- Most philosophers believe that creative abduction – where the observation is a surprise (like the double-slit experiment that suggested QM, or the GTR) – is not possible from inductive learning.

So, is AGI possible?

- **Must be**, because all processes in the brain can be simulated by an Universal Turing Machine.
- However, *inductive learning*, even if augmented by time-bounded deductive reasoning, cannot be a theory for AGI.
- There is more to *critical* and *ethical* thinking, let alone other *cognitive processes* and *sensorimotor perception*.
- *We have really made no real progress, need to cross some crucial epistemological barriers* (Deutsch, 1993 and continues)

Ultimately, there is nothing like some good theory